

Design decisions

I tried to keep it as close as possible to what the hints provided in the handout. I was a little confused with frozensets and how that worked with `initial_state`, but basically everything stems off of getting the state at which you are at by calling `initial_state`. Because our searches need to put states into explored sets and dictionaries, I represented the state as a hashable tuple containing the robot's (row, col) position and a frozenset of the dirty cells.

You use `is_goal` and `get_actions` to tell you what you need to do. If you get `True` from `is_goal`, that means there is no more dirt, but if not then we will use `get_actions` to tell us what our next possible moves are. `get_actions` enforces the canonical order (Up, Down, Left, Right, Clean) and ensures we don't try to clean a clean cell or walk into a wall. The result function is carefully written to return a brand new state tuple rather than mutating the existing one, so all the states sitting in the frontier don't get accidentally overwritten. `action_cost` just returns 1 because every action is determined to be one. Lastly, `render` goes through and checks each character in each row and sees if it is dirty or if there is a wall and gives that character the proper symbol.

For the search algorithms, I implemented DFS iteratively with an explicit stack. To ensure the stack explored nodes in the correct `ACTION_ORDER`, I pushed the successors onto the stack in `reversed()` order.

For A*, handling reopening was a critical design decision. I maintained a `best_g` dictionary to track the lowest cost-so-far to reach any given state. When generating children, if we find a cheaper path to a state we have already seen, we update it like this:

Python

```
if child not in best_g or new_g < best_g[child]:
    best_g[child] = new_g
    came_from[child] = (state, action)
    h = heuristic(child, problem)
    pq.push(child, new_g + h)
```

Because the provided `PriorityQueue` automatically replaces an item's priority instead of duplicating it, this efficiently handled reopening without bloating the frontier.

For IDA*, I used a bounded DFS approach with linear space complexity. I tracked the states on the current path in a `path_states` set to prevent immediate looping, and deepened the search iteratively based on the smallest f-cost that exceeded the current threshold.

Heuristic analysis

Heuristics are a very important part of this mini project because they are needed in A* and IDA* searches. They tell you what the estimated remaining cost is at any given state. We started with the Manhattan function which is just checking the distance between two points. On this grid, the true number of moves between two cells is always at least their Manhattan distance because obstacles can only force a detour, never a shortcut.

With h_0 we just return 0 because we want to use this in A* as our baseline. This turns A* into uniform-cost search so we have an experimental baseline.

h_1 Admissibility Argument: h_1 just checks to see how many more dirty cells are left based on the dirty set in the state by returning the length of the dirty set. It is completely admissible because each remaining dirty cell strictly requires its own CLEAN action. Because every action (including CLEAN) costs exactly 1, a state with k dirty cells will mathematically require a minimum cost of k just to execute the cleans. By ignoring the travel cost entirely, h_1 serves as a strict lower bound and will never overestimate the true remaining cost.

h_2 Admissibility Argument: h_2 checks Manhattan distance to the next dirty cell and then also returns that plus how many more dirty cells are left. If there is no dirt, it returns 0. This gives a lower bound because the robot must physically move to a dirty cell before it can perform any cleaning, and the true travel distance to reach that first cell is always at least the Manhattan distance to the nearest one, on top of requiring at least one clean action for every dirty cell remaining. To clean anything, the robot must physically move to at least one dirty cell first. The cheapest possible travel cost to start cleaning is the Manhattan distance to the nearest dirty cell. Once there, the robot still has to execute at least k CLEAN actions. Adding the absolute minimum travel cost to the absolute minimum cleaning cost guarantees that h_2 never overestimates the actual required steps, making it admissible and dominant over h_1 .

h_3 (Bonus) Admissibility Argument: For the bonus heuristic, I realized h_2 only ever looks at one dirty cell, but the robot still has to get to all the others. To find a cheap-to-compute lower bound on that remaining travel, I calculated the Minimum Spanning Tree (MST) weight connecting the robot's current position and all remaining dirty cells. Any valid path that visits all dirty cells forms a spanning walk across those points. Because the MST represents the absolute minimum edge weight required to connect a graph, and the edges are Manhattan distances (which already underestimate true grid distance), the MST weight is a guaranteed lower bound on the total travel required. Adding this to the number of dirty cells (the cleaning cost) gives an admissible heuristic that dominates.

Table from experiments.py

grid	algo	h	status	nodes / cost / time
------	------	---	--------	---------------------

g1_tiny	dfs	-	ok	16 nodes, cost 12, 0.00s
g1_tiny	astar	h0	ok	21 nodes, cost 7, 0.00s
g1_tiny	astar	h1	ok	20 nodes, cost 7, 0.00s
g1_tiny	astar	h2	ok	13 nodes, cost 7, 0.00s
g1_tiny	astar	h3	ok	8 nodes, cost 7, 0.00s
g1_tiny	idastar	h0	ok	95 nodes, cost 7, 0.00s
g1_tiny	idastar	h1	ok	52 nodes, cost 7, 0.00s
g1_tiny	idastar	h2	ok	27 nodes, cost 7, 0.00s
g1_tiny	idastar	h3	ok	8 nodes, cost 7, 0.00s
g2_open	dfs	-	ok	48 nodes, cost 36, 0.00s
g2_open	astar	h0	ok	98 nodes, cost 12, 0.00s
g2_open	astar	h1	ok	96 nodes, cost 12, 0.00s
g2_open	astar	h2	ok	76 nodes, cost 12, 0.00s
g2_open	astar	h3	ok	37 nodes, cost 12, 0.00s
g2_open	idastar	h0	ok	15586 nodes, cost 12, 0.09s
g2_open	idastar	h1	ok	3821 nodes, cost 12, 0.02s
g2_open	idastar	h2	ok	616 nodes, cost 12, 0.01s
g2_open	idastar	h3	ok	16 nodes, cost 12, 0.00s
g3_blocks	dfs	-	ok	69 nodes, cost 43, 0.00s
g3_blocks	astar	h0	ok	411 nodes, cost 15, 0.00s
g3_blocks	astar	h1	ok	331 nodes, cost 15, 0.00s

g3_blocks	astar	h2	ok	256 nodes, cost 15, 0.00s
g3_blocks	astar	h3	ok	37 nodes, cost 15, 0.00s
g3_blocks	idastar	h0	ok	29281 nodes, cost 15, 0.16s
g3_blocks	idastar	h1	ok	4463 nodes, cost 15, 0.03s
g3_blocks	idastar	h2	ok	2010 nodes, cost 15, 0.02s
g3_blocks	idastar	h3	ok	17 nodes, cost 15, 0.00s
g4_pillars	dfs	-	ok	110 nodes, cost 80, 0.00s
g4_pillars	astar	h0	ok	1160 nodes, cost 19, 0.01s
g4_pillars	astar	h1	ok	850 nodes, cost 19, 0.01s
g4_pillars	astar	h2	ok	510 nodes, cost 19, 0.01s
g4_pillars	astar	h3	ok	20 nodes, cost 19, 0.00s
g4_pillars	idastar	h0	ok	58097 nodes, cost 19, 0.29s
g4_pillars	idastar	h1	ok	8883 nodes, cost 19, 0.05s
g4_pillars	idastar	h2	ok	4174 nodes, cost 19, 0.04s
g4_pillars	idastar	h3	ok	19 nodes, cost 19, 0.00s
g5_rooms	dfs	-	ok	136 nodes, cost 92, 0.00s
g5_rooms	astar	h0	ok	2403 nodes, cost 25, 0.02s
g5_rooms	astar	h1	ok	2097 nodes, cost 25, 0.02s
g5_rooms	astar	h2	ok	1651 nodes, cost 25, 0.02s
g5_rooms	astar	h3	ok	162 nodes, cost 25, 0.00s
g5_rooms	idastar	h0	TIMEOUT	
g5_rooms	idastar	h1	ok	1802996 nodes, cost 25, 10.00s
g5_rooms	idastar	h2	ok	731040 nodes, cost 25, 6.85s
g5_rooms	idastar	h3	ok	1011 nodes, cost 25, 0.03s
g6_corridor	dfs	-	ok	195 nodes, cost 121, 0.00s
g6_corridor	astar	h0	ok	10873 nodes, cost 32, 0.08s

g6_corridor astar h1 ok 7665 nodes, cost 32, 0.06s

g6_corridor astar h2 ok 5797 nodes, cost 32, 0.06s

g6_corridor astar h3 ok 92 nodes, cost 32, 0.00s

g6_corridor idastar h0 TIMEOUT

g6_corridor idastar h1 TIMEOUT

g6_corridor idastar h2 TIMEOUT

g6_corridor idastar h3 ok 116 nodes, cost 32, 0.01s

Discussion

1. DFS node expansion vs. solution quality

DFS dives directly down the search tree, picking the first available action until it hits a goal. It doesn't explore breadth-wise or care about the cost, which is why it usually expands far fewer nodes (it just gets lucky and finds a path without exploring the rest of the possible tree). However, because it completely ignores action costs and shorter alternatives, the first path it finds is usually highly suboptimal. For example, on g6_corridor, DFS expanded only 195 nodes but returned a terrible path cost of 121, whereas the optimal cost is 32.

2. A node count from h0 to h1 to h2*

Looking at the data for A* on g4_pillars, the node count goes from 1,160 (h0) down to 850 (h1), and then down to 510 (h2). This perfectly matches what dominance predicts. Because h2 is strictly greater than or equal to h1 for every state, it provides a tighter lower bound on the true cost. This allows the A* algorithm to prune unpromising branches much earlier in the search tree, strictly reducing (or keeping equal) the number of nodes it has to expand.

3. IDA expansion vs. A and its use case IDA* expands vastly more nodes than A* (for example, on g4_pillars with h1, IDA* expanded 8,883 nodes compared to A*'s 850). This happens because IDA* does not keep a came_from dictionary or explored set across its iterations. Every time the threshold increases, it has to re-expand the upper levels of the search tree from scratch. However, someone would still use IDA* when memory is the primary bottleneck. A* has an exponential space complexity and will quickly crash if the queue gets too large. IDA* runs in linear space, making it the right choice for running search algorithms on hardware with very limited RAM (like small embedded devices or microcontrollers).

4. State space growth and "blowing up" A single state in this problem consists of the robot's coordinates (row, col) and the frozenset of remaining dirty cells. If we let N be the

number of passable cells in the grid and D be the initial number of dirty cells, the robot can be in N possible positions, and there are 2^D possible combinations of dirty cells remaining (since every dirt pile is either clean or dirty). This means the formula for the total number of reachable states is $N \times 2^D$. Adding just one dirty cell literally doubles the size of the state space. This exponential growth is why the search "blows up" and times out on grids with 7 or 9 dirty cells.

5. The hardest grid

Based on the experimental data, `g6_corridor` was the hardest grid. The main structural property making it so difficult was the sheer number of dirty cells (9 dirty cells). Because of the 2^D state space growth mentioned above, 9 dirty cells created $2^9 = 512$ different combinations of dirt subsets just for a single robot position. This caused a massive explosion in the state space, resulting in IDA* timing out (exceeding 60 seconds) for heuristics h_0 , h_1 , and h_2 , and forcing A* to expand over 10,000 nodes when uninformed.

6. The Roomba Question: Removing the Deterministic Actions assumption If we removed the "deterministic actions" assumption—meaning the robot might slip, ignore a command, or accidentally move left when told to move up—my implementation would completely break. My `result(state, action)` function assumes a 100% chance of landing in the exact calculated next coordinate. If actions were uncertain, a fixed path of moves generated by A* would become useless the moment the robot slipped. Instead of classic search, this becomes a Markov Decision Process (MDP). To solve it, we would need Reinforcement Learning techniques or Value Iteration to calculate a "policy" that tells the robot what to do from every possible state, rather than just returning a static list of actions.

Reflection

Getting the algorithms to behave exactly as expected took a lot of trial and error. One specific issue I ran into early on was with my result function. At first, I accidentally used regular sets to track the dirty cells and tried removing items from the set directly inside the result function. This mutated the state in place. Because Python assigns objects by reference, every single state sitting in my priority queue suddenly updated to show that cell as clean. My A* search completely broke and just spun out of control because all the states converged. I found this by printing out the states as they were pulled from the frontier on a tiny 3x3 grid and realizing the "dirty" sets were matching across entirely different branches of the tree. I fixed it by utilizing `frozenset` and subtracting the coordinate using the `-` operator to generate a completely new tuple.

Another major challenge was handling reopening in A*. Initially, my frontier size was blowing up because I was just blindly pushing states into the `PriorityQueue` without

updating the `best_g` dictionary properly. I had to go back and carefully ensure that I was only pushing a newly generated child into the queue if its new g -cost was strictly lower than what I had previously recorded in `best_g`. If I were to do this project again, I would write isolated unit tests just for the state transition logic and heuristic outputs before plugging them into the complex search loops. Diagnosing math logic errors deep inside an A* queue with hundreds of nodes was incredibly difficult, and having verified helper functions would have saved me a lot of debugging time.